



Technical University of Denmark

02244 Logic for security

Group 31

Project on Security Protocols

Group Members:

Deniz Isikli	s215818@student.dtu.dk
Benas Skripkiunas	s253140@student.dtu.dk
Abid Hasan	s252729@student.dtu.dk

16th March, 2026

Contents

1	Introduction	3
1.1	Security Goals	3
1.2	Modeling and Verification Tools	3
1.3	Experimental Setup	4
2	Protocol Design Progression	4
2.1	Week 2 – Initial Symmetric-Key Protocol	5
2.2	Week 3 – Introduction of an Identity Provider	5
2.3	Week 4 – Message Typing and Key Lookup	6
2.4	Week 5 – TLS-Style Secure Channels	6
3	Final Protocol Description	7
3.1	Protocol Overview	8
3.2	Final Protocol in AnB	8
3.3	Initial Knowledge	8
3.4	Security Goals	9
4	Security Analysis	9
4.1	Verification Results	10
4.2	Authentication Analysis	10
4.3	Secrecy Analysis	10
5	Special Tasks	11
5.1	Week 2 – Static Dolev-Yao Analysis	11
5.2	Week 3 – Lazy Intruder Executability	11
5.3	Week 4 – Type-Flaw Resistance	12

5.4	Week 5 – Modeling TLS Channels	12
6	Assumptions & Simplifications	12
6.1	Dolev-Yao Model	13
6.2	Bounded Session Analysis	13
6.3	Trusted Identity Provider	13
6.4	Password Modeling	13
6.5	Abstract Channel Modeling	14
7	Conclusion	14
8	Appendix	15
8.1	Logbook	15
8.1.1	Week 2	15
8.1.2	Week 3	18
8.1.3	Week 4	21
8.1.4	Week 5	24
8.1.5	Week 6	27
8.2	Group Contributions	28
8.3	Week 2 Protocol	28
8.4	Week 3 Protocol	29
8.5	Week 4 Protocol	29
8.6	Week 5 Protocol	30

1 Introduction

Security protocols are essential for enabling secure communication between distributed systems. They define how different parties exchange messages in order to achieve properties such as authentication, confidentiality, and integrity, even in the presence of an active adversary.^[2] Designing such protocols is challenging because subtle flaws in message structure, key management, or authentication mechanisms may allow an attacker to compromise the intended security guarantees.^[1]

In this project, we design and analyze a security protocol for a simplified photo service scenario. The system involves four parties: a user A , a storage server B , a photo service P , and an identity provider ip . The goal of the protocol is to allow the service p to retrieve data belonging to A from server B , while ensuring that the data remains confidential and that P can authenticate that the response originates from B . The identity provider is responsible for authenticating users and assisting in establishing trust between the different parties.

The protocol was developed iteratively over several weeks. Each iteration introduced new design elements and security considerations based on lectures and weekly tasks and special task. Initially, the protocol relied on symmetric keys and simple challenge-response mechanisms. Later versions incorporated an identity provider, improved message typing to prevent type-flaw attacks, and finally replaced explicit cryptographic constructions with TLS-like channel abstractions for communicating in secure channel. Throughout the development process, the protocols were modeled using Alice-and-Bob (AnB) notation and analyzed using the OFMC model checker.

1.1 Security Goals

The main security goals of the protocol are authentication and confidentiality. In particular, the protocol should ensure that the photo service P can authenticate that the returned data originates from server B , and that the data itself remains confidential between these two parties. Formally, the following goals are specified in the AnB models:

- P authenticates B on the parameters of the response, ensuring that the payload originates from B .
- The value *Payload* remains secret between B and P .

These goals are verified using automated protocol analysis with the OFMC model checker.

1.2 Modeling and Verification Tools

All protocols in this project were specified using Alice-and-Bob (AnB) notation and analyzed using the Open-Source Fixedpoint Model Checker (OFMC). AnB provides a high-level language for describing

security protocols, including agent knowledge, message exchanges, and desired security properties. The OFMC tool then translates this specification into a formal model and attempts to find attacks under the Dolev-Yao adversary model.

The Dolev-Yao model assumes a powerful adversary who can intercept, modify, and inject messages on the network, but cannot break cryptographic primitives.^[3] Security properties are therefore analyzed symbolically by exploring all possible protocol executions within the given bounds.

1.3 Experimental Setup

The protocol analysis was performed using the OFMC backend with a bounded number of protocol sessions. In particular, all protocol versions were tested using the following command:

```
./OFMC/linux/ofmc --numSess 2 Protocols/<protocol>.AnB  
OR  
./OFMC/mac/ofmc --numSess 2 Protocols/<Protocol>.AnB
```

The parameter `-numSess 2` limits the analysis to two concurrent protocol sessions. This allows the model checker to explore interactions between multiple protocol runs, which is often necessary to detect replay attacks or other interleaving attacks.

During verification, OFMC attempts to construct an execution trace that violates the specified security goals. If no such trace is found within the bounded search space, the tool reports:

```
SUMMARY :  
NO_ATTACK_FOUND  
DETAILS :  
BOUNDED_NUMBER_OF_SESSIONS
```

This result indicates that no attacks were discovered for the given number of sessions and under the assumptions of the symbolic model. The analysis therefore provides evidence that the specified authentication and secrecy goals hold within the explored execution space.

2 Protocol Design Progression

For this project, each week has been iteratively developed and analysed with the focus of completing the given task for the week, as well as attempting to finish the special task.^[4] The protocol evolved step by step, starting from a simple baseline design and gradually incorporating more realistic security mechanisms and modeling techniques. At each stage, the protocol was specified in AnB notation

and verified using OFMC in order to identify potential weaknesses and confirm whether the intended security properties were satisfied.

2.1 Week 2 – Initial Symmetric-Key Protocol

The first version of the protocol establishes a simple interaction between three parties: the user A , the storage server B , and the photo service P . In this initial design, it is assumed that symmetric keys are already shared between P and B , and between B and P . These keys are used to protect the confidentiality and integrity of the exchanged messages.

The protocol begins with P initiating a request to A by sending a nonce NP . This nonce serves as a freshness value that binds the subsequent messages to the current protocol execution.^[1] Agent A then forwards the request to server B , encrypting the message using the shared key between A and B . Server B responds with a challenge containing the original nonce NP together with a newly generated nonce NB . Agent A proves liveness by returning NB to B . After successful verification of this value, B releases the requested data to the service P , encrypted using the key shared between B and P .

In the AnB model, the protocol messages are represented using explicit format constructors in order to ensure clear message structure and avoid ambiguity in message interpretation.

This initial version provides a simple authentication and confidentiality mechanism. The use of nonces ensures freshness and prevents replay attacks within a single session, while symmetric encryption protects the exchanged data from unauthorized access.

However, this version relies on several simplifying assumptions. In particular, it assumes that all necessary symmetric keys are already distributed among the agents. In realistic systems, such key distribution is typically handled by a trusted identity provider or authentication service. Addressing this limitation motivates the changes introduced in the following week.

The complete AnB model for the Week 2 protocol is shown in Appendix 8.3.

2.2 Week 3 – Introduction of an Identity Provider

In Week 3, the protocol was extended to include an identity provider ip , which is responsible for authenticating the user A .^[4] Instead of relying solely on pre-shared symmetric keys between all parties, the user now authenticates through the identity provider using a shared password $pw(A, ip)$.

The protocol begins with A contacting ip and sending its identity together with a nonce NA and the shared password. Once the identity provider verifies the authentication, it forwards a request to server B containing the identity of A and the nonce NA . This message is encrypted using the symmetric key shared between ip and B , ensuring that only B can read and verify the request.

After receiving this request, server B interacts with the photo service P in order to establish a fresh

session. A new nonce NB is introduced in the communication between B and P , and the service responds with an additional nonce NP to confirm liveness. Finally, server B sends the protected data (*Payload*) to P , bound to the nonces NB and NP .

To ensure that messages cannot be misinterpreted during protocol execution, explicit message format constructors are used in the AnB specification to represent the different protocol messages.

This version introduces a more realistic authentication structure, where an identity provider is responsible for verifying users before access to protected resources is granted. The additional nonces further strengthen the freshness guarantees of the protocol.

The complete AnB model for the Week 3 protocol is shown in Appendix 8.4.

2.3 Week 4 – Message Typing and Key Lookup

The focus of Week 4 was to ensure that the protocol is resistant to type-flaw attacks^[5] and to remove the assumption that the user initially knows the public keys of all other agents. Instead, the user must obtain the public key of server B from the identity provider before the protocol proceeds.

To prevent type confusion between messages, explicit message formats were introduced using format constructors. Each protocol message is represented using a distinct constructor, ensuring that messages with different purposes cannot be mistakenly interpreted as each other.^[5] This structural separation prevents unification of incompatible messages and therefore eliminates the possibility of type-flaw attacks within the symbolic model.

Additionally, a key lookup step was introduced at the beginning of the protocol. Agent A first contacts the identity provider to request the public key of B . Once the key is received, the protocol continues with the authentication and authorization steps. This modification reflects a more realistic deployment scenario in which public keys are distributed by a trusted authority rather than assumed to be globally known.

The complete AnB model for the Week 4 protocol is shown in Appendix 8.5.

2.4 Week 5 – TLS-Style Secure Channels

In the final week, the protocol was modified to replace explicit cryptographic operations with secure communication channels modelled after TLS. Instead of directly specifying encryption and signatures, the protocol now uses the pseudonymous channel notation provided by OFMC.^[6]

These channels are denoted by the operators $*->$ and $->*$. They represent communication links where the receiving party can authenticate the sender's endpoint, which corresponds to server-side authentication in TLS. By using these channels, several explicit cryptographic constructions from the earlier protocol versions can be replaced by a simpler abstraction.

The overall structure of the protocol remains largely unchanged. Agent A authenticates with the identity provider using a password-based challenge-response mechanism. The identity provider then forwards the authorization to the photo service P , which interacts with the storage server B to retrieve the requested data. In addition, the identity provider produces signed authorization statements that bind the participating agents and session parameters. These signed statements are forwarded through the protocol so that the receiving parties can verify that the authorization originates from the trusted identity provider.

The final message containing the *Payload* is transmitted over a pseudonymous channel, ensuring that P can authenticate that the response originates from B while keeping the payload confidential between the two parties.

This final version provides a simplified representation of a system that uses TLS-like communication channels while maintaining the security properties developed in the earlier protocol iterations.

The complete AnB model for the Week 5 protocol is shown in Appendix 8.6.

3 Final Protocol Description

The final version of the protocol corresponds to the Week 5 design, where explicit cryptographic operations are replaced by TLS-style pseudonymous channels.^[6] This version maintains the security properties developed throughout the previous weeks while simplifying the protocol structure by modeling secure communication channels rather than individual cryptographic primitives.

The protocol involves four agents: the user A , the identity provider ip , the storage server B , and the photo service P . The user A authenticates to the identity provider using a password-based challenge-response mechanism. Once authentication is completed, the identity provider forwards the request to the photo service P , which then interacts with the storage server B in order to obtain the requested data.

The communication between the agents is modelled using pseudonymous channels. In OFMC, these channels are represented using the operators $*->*$.^[1] These operators model secure communication links where the receiver can authenticate the sender's endpoint, corresponding to server-side authentication in TLS. Using these channels allows the protocol to abstract away from explicit encryption mechanisms while preserving the intended security guarantees.

In addition, the identity provider issues signed authorization statements that bind together the identities of the agents and the session parameters. These signed statements are forwarded through the protocol to allow the receiving parties to verify that the authorization originates from the trusted identity provider.

3.1 Protocol Overview

The protocol proceeds in the following steps:

1. The user A initiates the protocol by contacting the identity provider ip and sending the identities of the involved parties together with a nonce NA .
2. The identity provider responds with a challenge nonce NID and a signed message containing the session information.
3. The user proves knowledge of the shared password by sending a hash of the password and the two nonces.
4. The identity provider forwards the validated request to the photo service P .
5. The photo service contacts the identity provider and receives a signed authorization statement containing the session parameters.
6. The photo service then forwards the request to the storage server B together with the signed authorization.
7. The storage server responds with a fresh nonce NB , after which the protocol continues to establish agreement on the session parameters.
8. Finally, server B returns the protected data (*Payload*) to the photo service P .

3.2 Final Protocol in AnB

The AnB specification of the final protocol is shown in Appendix 8.6.

3.3 Initial Knowledge

Each agent begins with a predefined set of knowledge required to execute the protocol.

- The user A knows the identities of the other agents, the shared password with the identity provider, the hash function used for authentication, and the public key of the identity provider.
- The identity provider ip knows the shared password and the hash function required to verify the authentication request, as well as its own key pair used to sign authorization statements.
- The storage server B and the photo service P know the identities of the participating agents and the public key of the identity provider used to verify signed authorization statements.

This distribution of knowledge reflects the roles of the different agents. The identity provider acts as the trusted authority responsible for authenticating the user and issuing authorization statements, while the storage server and the photo service participate in the data retrieval phase.

3.4 Security Goals

The final protocol specifies several authentication and secrecy goals.

First, the user A must be able to authenticate the identity provider ip with respect to the session parameters A, B, P, NA , and NID . This ensures that the authentication challenge originates from the trusted identity provider.

Second, the photo service P must be able to authenticate the identity provider on the parameters A, B, P, NA , and NP , ensuring that the authorization information was issued by the trusted identity provider.

Third, the storage server B authenticates the photo service P on the parameters A, NA, NP , and NB , ensuring that the request originates from the correct service.

Fourth, the photo service P authenticates the storage server B on the parameters A, NA, NP, NB , and $Payload$. This ensures that the returned data originates from the legitimate storage server and corresponds to the current protocol execution.

Finally, the value $Payload$ must remain secret between B and P . This ensures that the protected data cannot be learned by any other party, including the network intruder modeled in the Dolev-Yao adversary model.

Together, these goals guarantee that the agents correctly authenticate each other during the protocol execution while preserving the confidentiality of the protected resource.

4 Security Analysis

The security of the developed protocols was evaluated using the Open-Source Fixedpoint Model Checker (OFMC).^[1] Each protocol version was specified in Alice-and-Bob (AnB) notation and analyzed under the Dolev-Yao adversary model. In this model, the intruder has full control over the communication network and can intercept, modify, and inject messages, but cannot break cryptographic primitives.^[3]

All protocol versions were verified using bounded session analysis with the following command:

```
./OFMC/linux/ofmc --numSess 2 Protocols/<protocol>.AnB  
OR  
./OFMC/mac/ofmc --numSess 2 Protocols/<protocol>.AnB
```

The parameter `-numSess 2` allows OFMC to analyze protocol executions with two concurrent sessions. This configuration enables the detection of attacks involving message replay or interleaving between different protocol runs. For each protocol version, OFMC explores the reachable state space in search of execution traces that violate the specified authentication or secrecy goals.

4.1 Verification Results

For all protocol versions, the OFMC analysis reported that no attacks were found for the specified goals. The typical output returned by the model checker is shown below:

```
SUMMARY :  
  NO_ATTACK_FOUND  
DETAILS :  
  BOUNDED_NUMBER_OF_SESSIONS
```

This result indicates that within the explored execution space, OFMC was not able to construct an attack trace violating the defined security properties. In particular, the model checker was unable to derive the protected value *Payload* or to produce traces that break the specified authentication properties.

4.2 Authentication Analysis

Several authentication properties are specified in the final protocol model. These properties ensure that the different agents correctly verify the identity of the communicating parties during the execution of the protocol.

One of the primary goals is that the photo service *P* must be able to authenticate the storage server *B* when receiving the protected data. This property is specified in the AnB model as:

```
P authenticates B on A, NA, NP, NB, Payload
```

This authentication goal ensures that whenever *P* receives a message containing the values *NB* and *Payload*, together with the associated session parameters, it can be confident that the message originated from *B* during the same protocol execution.

4.3 Secrecy Analysis

The protocol also specifies a secrecy goal for the protected value *Payload*:

Payload secret between B , P

This property guarantees that only the storage server B and the photo service P are able to obtain the protected data. According to the OFMC analysis, the intruder cannot derive the value *Payload* from the messages exchanged during the protocol.

5 Special Tasks

Each week of the project included a specific additional task designed to analyze particular aspects of the protocol or to demonstrate certain modeling techniques.^[4] In the following subsections, we summarize how each of these tasks was addressed during the development of the protocol.

5.1 Week 2 – Static Dolev-Yao Analysis

The special task for Week 2 was to perform a static analysis of the protocol under the Dolev-Yao model assuming a passive intruder. In this setting, the intruder is able to observe all messages transmitted over the network but does not actively inject or modify messages.

In the initial protocol, the intruder can observe the first message sent from P to A , which contains the identities of the agents and the nonce NA . However, the remaining messages are encrypted using symmetric keys shared between the legitimate participants. Since the intruder does not possess the keys $sk(A, B)$ or $sk(B, P)$, it cannot decrypt these messages.

As a result, the intruder is unable to learn the nonce NB or the protected value *Payload*. Under the passive Dolev-Yao assumption, the protocol therefore preserves the confidentiality of the protected data.^{8.3}

5.2 Week 3 – Lazy Intruder Executability

The special task for Week 3 required demonstrating that the role of the photo service P is executable using the lazy intruder technique. This analysis shows that the messages required by the role can be constructed using only the information available from previous messages and the knowledge of the role.

In this scenario, the intruder is instantiated as the agent playing the role of P , while all other agents behave honestly. When server B sends the message containing $\{A, NA, NB\}_{sk(B, P)}$, the intruder acting as P possesses the key $sk(B, P)$ and can therefore decrypt the message and obtain the nonce NB . The intruder can then generate a fresh nonce NP and construct the response $\{NB, NP\}_{sk(B, P)}$, which satisfies the expected message structure for the protocol.

Since the intruder can construct all required outgoing messages of role P using the information obtained during the protocol execution, the role is considered executable according to the lazy intruder method.^{8.4}

5.3 Week 4 – Type-Flaw Resistance

The special task for Week 4 was to ensure that the protocol is resistant to type-flaw attacks. Such attacks occur when messages of different types can be interpreted as each other due to ambiguous message structures.

To prevent this issue, explicit format constructors were introduced for each message in the protocol. Instead of using simple concatenation of terms, every message is wrapped in a unique format function (e.g., $f0$, $f1$, $f2$, etc.). This ensures that each message type has a distinct structure.

Under the symbolic model used by OFMC, two messages can only unify if they share the same constructor. Since all protocol messages use different format constructors, messages of different types cannot unify with each other. This guarantees that the protocol is resistant to type-flaw attacks.^{8.5}

5.4 Week 5 – Modeling TLS Channels

The special task for Week 5 was to replace as much explicit cryptography as possible with TLS-style communication channels. This was achieved using the pseudonymous channel notation supported by OFMC.

The operator $*->*$ was used to model channels where the receiving party can authenticate the sender's endpoint. This abstraction captures the behavior of TLS connections where the server is authenticated while the communication channel provides confidentiality and integrity.

By replacing explicit encryption with pseudonymous channels, the protocol becomes simpler while still maintaining the intended authentication and secrecy guarantees. The resulting protocol therefore models a TLS-like system where secure channels are assumed to provide the necessary protection for the exchanged messages.^{8.6}

6 Assumptions & Simplifications

The protocol analysis performed in this project relies on several modeling assumptions and simplifications. These assumptions are common in symbolic protocol analysis and are necessary in order to make automated verification tractable.

6.1 Dolev-Yao Model

All protocols were analyzed under the Dolev-Yao adversary model. In this model, the intruder is assumed to have complete control over the network and can intercept, modify, and inject messages arbitrarily. However, cryptographic primitives such as encryption and hashing are assumed to be perfect. This means that the intruder cannot break encryption or invert hash functions unless the corresponding keys are already known.

While this abstraction is useful for formal analysis, it does not capture real-world cryptographic weaknesses such as side-channel attacks or flaws in cryptographic implementations.

6.2 Bounded Session Analysis

The protocol verification was performed using OFMC with the parameter `-numSess 2`, meaning that the analysis considered at most two concurrent protocol sessions. This bounded analysis is typically sufficient to detect common replay or interleaving attacks, but it does not constitute a formal proof of security for an unbounded number of sessions.

Nevertheless, bounded verification is widely used in practice and provides strong evidence that the protocol design does not contain obvious flaws.

6.3 Trusted Identity Provider

The identity provider ip is assumed to be fully trusted and behaves honestly throughout the protocol execution. In particular, it is assumed that the identity provider correctly verifies authentication requests and only forwards valid authorization messages.

In a real deployment, compromise of the identity provider would undermine the entire authentication infrastructure, which is outside the scope of the model considered in this analysis.

6.4 Password Modeling

The shared password between the user A and the identity provider is modeled as an abstract value $pwA(A, ip)$. The protocol assumes that this value is not known to the intruder and cannot be guessed. In practice, passwords may be weak or vulnerable to guessing attacks, which are not explicitly modeled in the symbolic analysis.

The use of hashing in the authentication step is intended to represent a simple challenge-response mechanism rather than a fully realistic password authentication protocol.

6.5 Abstract Channel Modeling

In the final protocol version, TLS-style communication channels were modeled using pseudonymous channel notation. These channels abstract away the underlying cryptographic operations and assume that the channel provides confidentiality, integrity, and server authentication.

While this abstraction simplifies the protocol model, it does not represent the full complexity of real TLS deployments. In particular, aspects such as certificate validation, session negotiation, and key exchange are not explicitly modeled.

7 Conclusion

In this project, we designed and analyzed a security protocol for a simplified photo service scenario involving a user A , a storage server B , a photo service P , and an identity provider ip . The protocol was developed iteratively across multiple stages, with each week introducing additional security features and modeling techniques.

The initial protocol version established a basic authenticated interaction using symmetric keys and nonces to ensure freshness and confidentiality. Subsequent iterations improved the design by introducing an identity provider for authentication, refining message structures to prevent type-flaw attacks, and eventually modeling TLS-style communication channels using pseudonymous channels provided by OFMC.

Throughout the development process, all protocol versions were formally specified using Alice-and-Bob (AnB) notation and verified using the Open-Source Fixedpoint Model Checker (OFMC). The verification results showed that the protocols satisfy the specified authentication and secrecy goals within the bounded session analysis. In particular, the final protocol ensures that the photo service P can authenticate the storage server B and that the protected data (*Payload*) remains confidential between these two parties.

While the analysis relies on several assumptions, such as the Dolev-Yao model of perfect cryptography and bounded session verification, the results provide strong evidence that the protocol design achieves the intended security properties. Overall, the project demonstrates how iterative design combined with formal verification tools can be used to develop and analyze security protocols in a systematic and reliable manner.

8 Appendix

8.1 Logbook

8.1.1 Week 2

Informal Outline

What has changed to previous version? What is the idea? In this week, we refined the initial protocol design and produced the first concrete protocol model. Instead of modeling a complex authorization structure, we focus on a simplified interaction between the three parties *A*, *B*, and *P*. The goal is to allow the photo book service *P* to obtain data stored at server *B* that belongs to *A*.

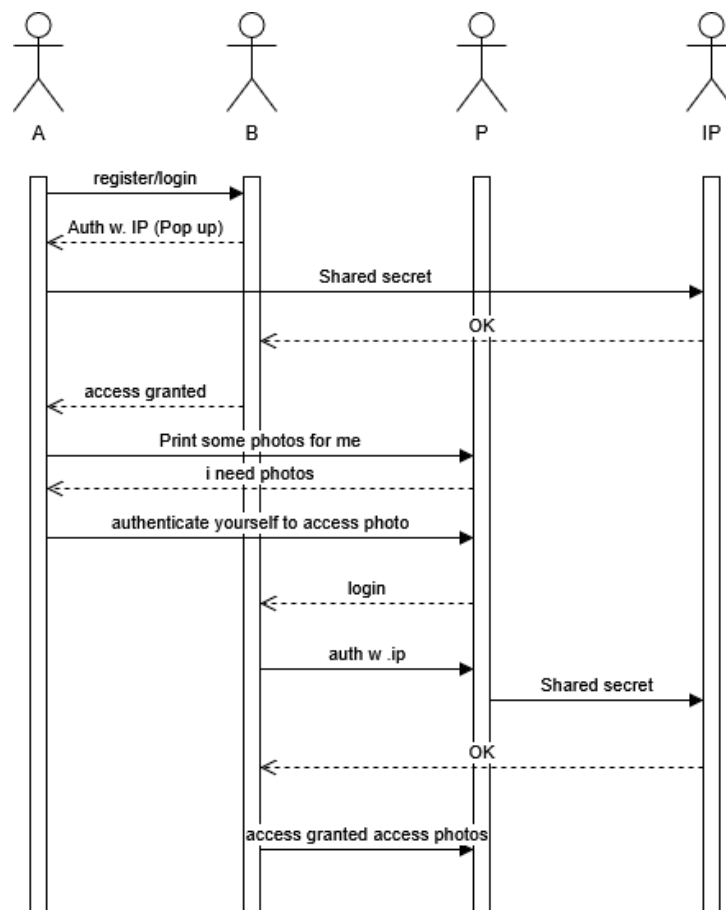


Figure 1: Initial delegation model (Week 2 snapshot).

The protocol begins with P initiating a request to A containing a nonce NA . This nonce serves as a freshness value that links the subsequent messages of the protocol. Agent A then forwards the request to B , encrypted using the shared symmetric key between A and B .

Server B responds with a challenge containing both the received nonce NA and a newly generated nonce NB , again protected with the shared key. Agent A proves liveness by returning NB to B . After successful verification, B releases the requested data to P , encrypted under the shared key between B and P .

Compared to the very first sketch of the protocol, this version introduces explicit message formats. These format constructors ensure that different messages have distinct structures, which prevents confusion between protocol messages and strengthens the model against type-flaw attacks.

The main idea of this version is therefore to establish a simple authenticated exchange using symmetric keys, nonces, and typed messages.

First AnB Model

Modeling considerations (assumptions and goals). The protected data is modeled as the value *Payload*. Two symmetric keys are assumed: one shared between A and B , and one shared between B and P . These keys are represented using the function $sk(\cdot, \cdot)$.

The protocol uses two nonces, NA and NB , to guarantee freshness and to bind the request and response phases together. Message structures are represented using format constructors $f1$ – $f5$, which replace simple concatenation and ensure that each message has a unique type.

We assume perfect cryptography under the Dolev–Yao model.

Protocol: PhotoBook_W2

Types:

Agent A, P, B ;
Number $NA, NB, NP, \text{Payload}$;
Function sk ;

Knowledge:

$A: A, B, P, sk(A, B)$;
 $B: A, B, P, sk(A, B), sk(B, P)$;
 $P: A, B, P, sk(B, P)$;

Actions:

$P \rightarrow A: P, B, NP$
 $A \rightarrow B: \{ | A, B, P, NA, NP | \}_{sk(A, B)}$

B → A: { | NP, NA, NB | }_{sk(A,B)}
A → B: { | NB | }_{sk(A,B)}
B → P: { | P, B, NB, NP, Payload | }_{sk(B,P)}

Goals:

P authenticates B on NB, NP, Payload;
B authenticates A on P, NA, NB;
Payload secret between B, P;

Special Task: Static Dolev–Yao Analysis (Passive Intruder)

We analyze the protocol assuming that the intruder only observes network traffic and does not inject messages.

The intruder observes the following transmissions:

$(P, B, NP), \{ | A, P, B, NA, NP | \}_{sk(A,B)}, \{ | NP, NA, NB | \}_{sk(A,B)}, \{ | NB | \}_{sk(A,B)}, \{ | P, B, NP, NB, Payload | \}_{sk(B,P)}.$

Under the Dolev–Yao assumptions, the intruder can learn the plaintext information contained in the first message, namely the identities of the communicating agents and the nonce *NA*.

However, all subsequent messages are encrypted using symmetric keys that the intruder does not possess. Since the intruder does not know $sk(A, B)$ or $sk(B, P)$, it cannot decrypt the protected messages. Consequently, the intruder cannot obtain the nonce *NB* or the protected data *Payload*.

Furthermore, the intruder cannot generate valid encrypted messages under these keys, since symmetric encryption is assumed to be perfect in the symbolic model. Therefore, the intruder cannot impersonate any of the agents or derive the protected information in the passive setting.

Problems and Reflections

The protocol was analyzed using OFMC with two sessions. The model checker reports that no attack was found for the specified goals.

Tool	OFMC (Open-Source Fixedpoint Model-Checker 2024)
Command	<code>./ofmc -numSess 2 Protocols/Week2.AnB</code>
Result	NO_ATTACK_FOUND
Goal	as specified
Sessions	BOUNDED_NUMBER_OF_SESSIONS
Time	72.267 seconds

This indicates that the protocol achieves the intended authentication and secrecy goals within the bounded session analysis.

The main improvements introduced in this version include:

- Use of symmetric encryption between A and B , and between B and P .
- Introduction of nonces NA and NB to ensure freshness of the protocol run.
- Authentication guarantees between the agents as defined in the protocol goals.
- Protection of the resource data (*Payload*) through encryption between B and P .

Despite these improvements, this model still relies on simplifying assumptions. In particular, symmetric keys are assumed to be pre-shared between the agents, and no identity provider is involved. This deviates from the intended scenario described in the project specification, where authentication is delegated through a trusted identity provider.

This version therefore serves as a simplified baseline protocol, which will be extended in Week 3 to incorporate the identity provider and a more realistic authentication mechanism.

8.1.2 Week 3

Informal Outline

What has changed to previous version? What is the idea? In this week, the protocol was extended to incorporate an identity provider ip , as required by the assignment. Unlike the previous version, agent A no longer relies solely on pre-shared symmetric keys with the other parties for authorization decisions. Instead, A authenticates herself to the identity provider using a shared password $pw(A, ip)$.

The protocol begins with A contacting the identity provider and sending her identity together with a nonce NA and the shared password. This step models the authentication of A toward the identity provider. Once authenticated, the identity provider forwards a request to the resource server B containing the identity of A and the nonce NA . This message is encrypted using the symmetric key shared between ip and B .

Server B then communicates with the photo book service P to establish a fresh session. It sends the identity of A , the original nonce NA , and a new nonce NB to P . The service P responds with another nonce NP to confirm liveness. Finally, once the handshake is completed, B sends the protected data (*Payload*) to P , bound to the nonces NB and NP .

Compared to Week 2, the protocol now models the presence of a trusted identity provider that authenticates A before access to B is granted. The design also introduces an additional nonce NP to ensure freshness in the interaction between B and P .

AnB Model

Modeling considerations (assumptions and goals). The model assumes perfect cryptography under the Dolev–Yao model. Agent A shares a password with the identity provider, modeled as the function $pw(A, ip)$. This password is used only for authentication and not as an encryption key, as required by the assignment.

The identity provider and server B share a symmetric key $sk(ip, B)$, while server B and the photo service P share a symmetric key $sk(B, P)$. These keys protect the confidentiality and integrity of the exchanged messages.

Three nonces (NA , NB , and NP) are used to guarantee freshness and bind the different steps of the protocol together. Message structures are represented using format constructors $f1$ – $f5$.

Protocol: PhotoBook_W3

Types:

Agent A , ip , B , P ;
Number NA , NB , NP , Payload;
Function sk , pk , pw ;

Knowledge:

A : A , ip , B , P , $pk(ip)$, $pw(A, ip)$;
 ip : A , ip , B , P , $pk(ip)$, $pw(A, ip)$, $sk(ip, B)$;
 B : A , ip , B , P , $pk(ip)$, $sk(ip, B)$, $sk(B, P)$;
 P : A , ip , B , P , $pk(ip)$, $sk(B, P)$;

Actions:

$A \rightarrow ip$: A , NA , $pw(A, ip)$
 $ip \rightarrow B$: $\{ | A, NA | \} sk(ip, B)$
 $B \rightarrow P$: $\{ | A, NA, NB | \} sk(B, P)$
 $P \rightarrow B$: $\{ | NB, NP | \} sk(B, P)$
 $B \rightarrow P$: $\{ | NB, NP, Payload | \} sk(B, P)$

Goals:

B authenticates P on NB, NP ;
 P authenticates B on NB, NP , Payload;
Payload secret between B , P ;

Special Task: Lazy Intruder (Role P Executability)

To demonstrate executability of the role P , we instantiate P with the intruder and assume all other agents behave honestly. According to the lazy intruder method, the intruder can generate any message that can be constructed from previously received messages and available functions.

Step 1: B sends the following message to the intruder playing the role of P :

$$B \rightarrow i : \{A, NA, NB\}_{sk(B,P)}$$

Step 2: Since the intruder is playing the role P , it is assumed to possess the key $sk(B, P)$ and can therefore decrypt the message and obtain NB .

Step 3: The intruder generates a fresh nonce NP .

Step 4: The intruder constructs the required response message:

$$i \rightarrow B : \{NB, NP\}_{sk(B,P)}$$

Step 5: After receiving this message, B proceeds with the final step of the protocol and sends:

$$B \rightarrow i : \{NB, NP, Payload\}_{sk(B,P)}$$

Conclusion: All outgoing messages of role P can be constructed using information obtained from previous messages and the shared key $sk(B, P)$. Therefore, the intruder can successfully execute the role of P , satisfying the Week 3 requirement.

Problems and Reflections

The protocol was analyzed using OFMC with two sessions. The model checker reports that no attacks were found for the specified goals.

Tool	OFMC (Open-Source Fixedpoint Model-Checker 2024)
Command	<code>./ofmc -numSess 2 Protocols/Week3.AnB</code>
Result	NO_ATTACK_FOUND
Goal	as specified
Sessions	BOUNDED_NUMBER_OF_SESSIONS
Time	79.980 seconds

This indicates that, under the current model and assumptions, the protocol satisfies both the authentication goal for P and the secrecy goal for the protected data (*Payload*).

The main improvements compared to Week 2 include:

- Introduction of an identity provider ip that authenticates A using a shared password.
- Separation of responsibilities between authentication (handled by ip) and data access (handled by B).
- NP to ensure freshness in the interaction between B and P .
- Explicit modeling of symmetric encryption between the different parties.

Despite these improvements, the protocol still relies on simplifying assumptions. In particular, the password is transmitted directly in the authentication step and is treated as an abstract value rather than a guessable secret. These aspects will be refined in later weeks when additional modeling techniques are introduced.

8.1.3 Week 4

Informal Outline

What has changed compared to the previous version? What is the idea?

In Week 4 the protocol was further refined to introduce explicit typing of messages and to remove the assumption that agent A initially knows the public keys of all other parties. Instead, A now retrieves the public key of the resource server B from the identity provider ip . This reflects a more realistic deployment scenario where a trusted authority distributes public keys.

The protocol therefore begins with a lookup step where A requests the public key of B from the identity provider. After receiving the key, A proceeds with the authentication step using the shared password $pw(A, ip)$ and a fresh nonce NA . Once authenticated, the identity provider forwards the request to the resource server B , which then interacts with the photo service P to provide the requested resource.

Compared to Week 3, the main conceptual change is the introduction of explicit message typing. Instead of simple concatenation of message components, each message now uses a dedicated format constructor $f_i(\cdot)$. This prevents different messages from being interpreted as each other and therefore protects the protocol against type-flaw attacks.

AnB Model

Modeling considerations (assumptions and goals).

The protocol is modeled under the Dolev–Yao attacker model, assuming perfect cryptography. The identity provider ip is modeled as a trusted party that knows the public keys of all agents. Agent A initially knows only the public key of the identity provider and must therefore request the public key of B before continuing the protocol.

Message structures are represented using the format constructors f_0 – f_6 . These constructors replace concatenation and provide explicit typing for each message. The symmetric keys $sk(ip, B)$ and $sk(B, P)$ are used to protect communication between the identity provider and the resource server, and between the server and the photo service.

Protocol: PhotoBook_W4

Types:

Agent A, ip, B, P ;
Number $NA, NB, NP, Payload$;
Function sk, pw, pk ;
Format $f_0, f_1, f_2, f_3, f_4, f_5, f_6$;

Knowledge:

A : $A, ip, B, P, pw(A, ip), pk(ip), f_0, f_1, f_2$;
 ip : $A, ip, B, P, pw(A, ip), pk(ip), pk(B), sk(ip, B), f_0, f_1, f_2, f_3$;
 B : $A, ip, B, P, sk(ip, B), sk(B, P), f_3, f_4$;
 P : $A, ip, B, P, sk(B, P), f_4, f_5, f_6$;

Actions:

$A \rightarrow ip$: $f_0(A, B)$
 $ip \rightarrow A$: $f_1(B, pk(B))$
 $A \rightarrow ip$: $f_2(A, NA, pw(A, ip))$
 $ip \rightarrow B$: $\{|f_3(A, NA)|\}_{sk(ip, B)}$
 $B \rightarrow P$: $\{|f_4(A, NA, NB)|\}_{sk(B, P)}$
 $P \rightarrow B$: $\{|f_5(NB, NP)|\}_{sk(B, P)}$
 $B \rightarrow P$: $\{|f_6(NB, NP, Payload)|\}_{sk(B, P)}$

Goals:

B authenticates P on NB, NP ;
 P authenticates B on $NB, NP, Payload$;
Payload secret between B, P ;

Special Task: Type-Flaw Resistance

To demonstrate that the protocol is type-flaw resistant, we analyze the set of submessage patterns (SMP) introduced by the format constructors. The SMP messages in the protocol are:

$f_0(A, B), f_1(B, pk(B)), f_2(A, NA, pw(A, ip)), f_3(A, NA), f_4(A, NA, NB), f_5(NB, NP), f_6(NB, NP, Payload)$.

Each message is constructed using a unique format constructor (f_0 – f_6). In the symbolic model used by OFMC, two terms can only unify if they share the same top-level constructor. Since each message in the protocol uses a distinct constructor, messages of different types cannot unify.

Therefore, for any two non-variable messages in the SMP, a unifier can only exist if both messages have the same constructor and therefore the same type. This satisfies the definition of type-flaw resistance given in the lecture.

Problems and Reflections

The protocol was analyzed using OFMC with two sessions. The verification result is summarized in Table 1.

Property	Result
Input Protocol	Protocols/Week4.AnB
Verification Result	NO_ATTACK_FOUND
Goal Evaluation	As specified
Session Bound	2 sessions
Backend	Open-Source Fixedpoint Model-Checker 2024
Execution Time	996172 ms

Table 1: OFMC verification result for the Week 4 protocol.

The analysis shows that no attack was found for the specified authentication and secrecy goals within the bounded session analysis.

The main improvements introduced in this version are:

- Explicit typing of protocol messages using format constructors.
- Removal of the assumption that A initially knows the public key of B .
- Introduction of a public key lookup step via the identity provider.
- Structural protection against type-flaw attacks.

This version represents the fully typed implementation of the protocol and prepares the design for the next step, where explicit cryptographic operations are replaced by secure communication channels.

8.1.4 Week 5

Informal Outline

What has changed to previous version? What is the idea?

In this week, the protocol was adapted to model secure communication channels rather than relying solely on explicit cryptographic operations. The goal is to represent TLS-like communication where only the server side is authenticated. This is modeled using the pseudonymous channel notation provided by OFMC.

Compared to Week 4, several explicit cryptographic constructions are replaced by secure channel abstractions. These channels are denoted using the $* \rightarrow *$ operator and its variants. The notation indicates that the communication occurs over a pseudonymous channel where the receiver can authenticate the sender's endpoint, similar to how TLS authenticates the server in practice.

The overall protocol logic remains similar. Agent A first authenticates with the identity provider ip using the shared password. Once the identity provider has validated the request, it forwards the authorization to the photo service P . The service then interacts with the storage server B in order to obtain the requested data. The identity provider also provides signed statements that bind the authorization information, which are forwarded through the protocol to maintain trust in the authorization chain.

AnB Model

Modeling considerations (assumptions and goals). The model assumes that the pseudonymous channel notation represents TLS-like communication where the receiving party can authenticate the sender's endpoint but does not necessarily obtain freshness guarantees.

The shared password between A and the identity provider is modeled as the constant $pwA(A, ip)$. Authentication is represented through a hash computation using the function $h(\cdot)$. The identity provider issues signed statements using its private key $inv(pk(ip))$, which allows other parties to verify that the authorization information originates from the trusted identity provider.

The data returned by the storage server B is modeled as the value *Payload*. Several authentication goals are specified in order to ensure that the agents correctly authenticate each other during the protocol execution, while also guaranteeing confidentiality of the payload.

Protocol: PhotoBook_W5

Types:

Agent A, B, P, ip ;

```
Number NA, NID, NP, NB, Payload;
Function h, pk;
Const pwA;
```

Knowledge:

```
A: A, B, P, ip, pwA(A,ip), h, pk(ip);
B: A, B, P, ip, pk(ip);
P: A, B, P, ip, pk(ip);
ip: A, B, P, ip, pwA(A,ip), h, pk(ip), inv(pk(ip));
```

Actions:

```
[A] *->* ip : A,P,B,NA
ip *->* [A] : {ip,A,P,B,NA,NID}inv(pk(ip))
[A] *->* ip : A,NID,h(pwA(A,ip),NID,NA)
ip *->* P : ip,A,P,B,NA
P *->* ip : P,A,B,NA,NP
ip *->* P : {ip,A,P,B,NA,NP}inv(pk(ip))
P *->* B : P,B,A,NA,NP,{ip,A,P,B,NA,NP}inv(pk(ip))
B *->* P : B,P,A,NA,NP,NB
P *->* B : P,B,A,NA,NP,NB,{ip,A,P,B,NA,NP}inv(pk(ip))
B *->* P : B,P,A,NA,NP,NB,Payload
```

Goals:

```
A authenticates ip on A,B,P,NA,NID;
P authenticates ip on A,B,P,NA,NP;
B authenticates P on A,NA,NP,NB;
P authenticates B on A,NA,NP,NB,Payload;
Payload secret between B,P;
```

Special Task: Modeling TLS Channels

The pseudonymous channel notation of OFMC was used to model TLS-like channels. In this abstraction, the receiver of a message can authenticate the sender's endpoint, which corresponds to server-side authentication in TLS.

For example, the message

$$[A] * \rightarrow^* ip$$

represents a communication channel where the identity provider can authenticate that the message originates from *A*. Similarly, messages such as

$$ip * \rightarrow^* P$$

and

$$B* \rightarrow^* P$$

represent channels where the receiving party authenticates the sender.

By using this notation, the protocol replaces several explicit cryptographic constructions from previous versions. Instead of explicitly modeling encryption or signatures, the security guarantees are provided by the channel abstraction combined with signed authorization statements from the identity provider.

Problems and Reflections

The protocol was analyzed using OFMC with two sessions. The analysis reports the following result:

Tool	OFMC (Open-Source Fixedpoint Model-Checker 2024)
Command	<code>./ofmc -numSess 2 Protocols/Week5Rev.AnB</code>
Result	NO_ATTACK_FOUND
Goal	as specified
Sessions	BOUNDED_NUMBER_OF_SESSIONS
Time	1062.748 seconds

This indicates that the authentication and secrecy goals are satisfied under the bounded session analysis.

The main improvements introduced in this version include:

- Replacement of several explicit cryptographic constructions with pseudonymous TLS-style channel abstractions.
- Modeling of server-side authentication through OFMC channel notation.
- Inclusion of signed authorization statements issued by the trusted identity provider.
- Stronger authentication goals between the agents involved in the protocol.

Although pseudonymous channels do not guarantee message freshness by themselves, the protocol uses nonces (NA , NID , NP , and NB) to bind protocol steps together and maintain a consistent session structure.

8.1.5 Week 6

Special Task: Verifying with Guessable Secrets

In the protocol, the password shared between A and the identity provider ip is a human-memorable secret and therefore potentially guessable. To verify that the protocol remains secure under this assumption, the password is modeled as a guessable secret in the AnB specification by adding the following goal:

Goals:

```
...  
pwA(A,ip) guessable secret between A,ip
```

This instructs OFMC to consider the possibility that the intruder may attempt to guess the password during protocol execution.

In the protocol design, the password is never transmitted directly. Instead, it is used inside a hash computation together with the nonces NA and NID :

$$h(pwA(A, ip), NID, NA)$$

Since both nonces are fresh and unpredictable values, the intruder cannot verify password guesses offline without interacting with the protocol participants.

Additionally, the use of fresh nonces ensures that each protocol run produces a different hash value even when the same password is used. This prevents the intruder from precomputing password guesses or reusing observations from previous protocol sessions, thereby limiting the effectiveness of guessing attacks in the symbolic model.

After adding the guessable secret goal, the protocol was verified again using OFMC, and the analysis still reports:

SUMMARY:

NO_ATTACK_FOUND

DETAILS:

BOUNDED_NUMBER_OF_SESSIONS

This confirms that the protocol remains secure even when the password between A and the identity provider is modeled as a guessable secret.

8.2 Group Contributions

	Deniz Isikli	Benas Skripkiunas	Abid Hasan
Protocol Implementation (AnB)			
Week 2 Protocol	✓	✓	✓
Week 3 Protocol	✓	✓	✓
Week 4 Protocol	✓	✓	✓
Week 5 Protocol	✓	✓	✓
Week 6 Verification	✓	✓	✓
Report Documentation			
Introduction	✓	✓	✓
Protocol Design Progression	✓	✓	✓
Final Protocol Description	✓	✓	✓
Security Analysis	✓	✓	✓
Special Tasks	✓	✓	✓
Assumptions & Simplifications	✓	✓	✓
Conclusion	✓	✓	✓

Table 2: Contribution overview of group members for protocol implementation and report writing.

8.3 Week 2 Protocol

Protocol: PhotoBook_W2

Types:

Agent A, P, B;
Number NA, NB, NP, Payload;
Function sk;

Knowledge:

A: A, B, P, sk(A,B);
B: A, B, P, sk(A,B), sk(B,P);
P: A, B, P, sk(B,P);

Actions:

P → A: P, B, NP
A → B: { | A,B, P, NA, NP | } sk(A,B)

B → A: { | NP, NA, NB | }_{sk(A,B)}
A → B: { | NB | }_{sk(A,B)}
B → P: { | P, B, NB, NP, Payload | }_{sk(B,P)}

Goals:

P authenticates B on NB, NP, Payload;
B authenticates A on P, NA, NB;
Payload secret between B, P;

8.4 Week 3 Protocol

Protocol: PhotoBook_W3

Types:

Agent A, ip, B, P;
Number NA, NB, NP, Payload;
Function sk, pk, pw;

Knowledge:

A: A, ip, B, P, pk(ip), pw(A,ip);
ip: A, ip, B, P, pk(ip), pw(A,ip), sk(ip,B);
B: A, ip, B, P, pk(ip), sk(ip,B), sk(B,P);
P: A, ip, B, P, pk(ip), sk(B,P);

Actions:

A → ip: A, NA, pw(A,ip)
ip → B: { | A, NA | }_{sk(ip,B)}
B → P: { | A, NA, NB | }_{sk(B,P)}
P → B: { | NB, NP | }_{sk(B,P)}
B → P: { | NB, NP, Payload | }_{sk(B,P)}

Goals:

B authenticates P on NB, NP;
P authenticates B on NB, NP, Payload;
Payload secret between B, P;

8.5 Week 4 Protocol

Protocol: PhotoBook_W4

Types:

Agent A, ip, B, P;
Number NA, NB, NP, Payload;
Function sk, pw, pk;
Format f0, f1, f2, f3, f4, f5, f6;

Knowledge:

A: A, ip, B, P, pw(A,ip), pk(ip), f0, f1, f2;
ip: A, ip, B, P, pw(A,ip), pk(ip), pk(B), sk(ip,B), f0, f1, f2, f3;
B: A, ip, B, P, sk(ip,B), sk(B,P), f3, f4;
P: A, ip, B, P, sk(B,P), f4, f5, f6;

Actions:

A -> ip: f0(A,B)
ip -> A: f1(B,pk(B))
A -> ip: f2(A,NA,pw(A,ip))
ip -> B: {|f3(A,NA)|}sk(ip,B)
B -> P: {|f4(A,NA,NB)|}sk(B,P)
P -> B: {|f5(NB,NP)|}sk(B,P)
B -> P: {|f6(NB,NP,Payload)|}sk(B,P)

Goals:

B authenticates P on NB,NP;
P authenticates B on NB, NP, Payload;
Payload secret between B, P;

8.6 Week 5 Protocol

Protocol: PhotoBook_W5

Types:

Agent A, B, P, ip;
Number NA, NID, NP, NB, Payload;
Function h, pk;
Const pwA;

Knowledge:

A: A, B, P, ip, pwA(A,ip), h, pk(ip);
B: A, B, P, ip, pk(ip);
P: A, B, P, ip, pk(ip);
ip: A, B, P, ip, pwA(A,ip), h, pk(ip), inv(pk(ip));

Actions:

```
[A] *->* ip : A,P,B,NA
ip  *->* [A] : {ip,A,P,B,NA,NID}inv(pk(ip))
[A] *->* ip : A,NID,h(pwA(A,ip),NID,NA)
ip  *->* P  : ip,A,P,B,NA
P   *->* ip : P,A,B,NA,NP
ip  *->* P  : {ip,A,P,B,NA,NP}inv(pk(ip))
P   *->* B  : P,B,A,NA,NP,{ip,A,P,B,NA,NP}inv(pk(ip))
B   *->* P  : B,P,A,NA,NP,NB
P   *->* B  : P,B,A,NA,NP,NB,{ip,A,P,B,NA,NP}inv(pk(ip))
B   *->* P  : B,P,A,NA,NP,NB,Payload
```

Goals:

```
A authenticates ip on A,B,P,NA,NID;
P authenticates ip on A,B,P,NA,NP;
B authenticates P on A,NA,NP,NB;
P authenticates B on A,NA,NP,NB,Payload;
Payload secret between B,P;
```


References

- [1] Sebastian Mödersheim. Protocol security verification tutorial. <https://www2.imm.dtu.dk/~samo/OFMC-tutorial.pdf>, 2018. n.d.
- [2] Sebastian Mödersheim. Modeling protocols - alice and bob. <https://learn.inside.dtu.dk/d21/1e/lessons/296254/topics/1158342>, 2026. n.d.
- [3] Sebastian Mödersheim. Modeling protocols - dolev and yao. <https://learn.inside.dtu.dk/d21/1e/lessons/296254/topics/1162484>, 2026. n.d.
- [4] Sebastian Mödersheim. Project on security protocols. <https://learn.inside.dtu.dk/d21/1e/lessons/296254/topics/1162334>, 2026. n.d.
- [5] Sebastian Mödersheim. Secure implementation and typing. <https://learn.inside.dtu.dk/d21/1e/lessons/296254/topics/1167779>, 2026. n.d.
- [6] Sebastian Mödersheim. Security protocols channels and composition. <https://learn.inside.dtu.dk/d21/1e/lessons/296254/topics/1171211>, 2026. n.d.